

第7章 文件与 I/O

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握字节流与字符流两大家族,理解"流 = 数据的单向管道"
- 掌握 FileInputStream/FileOutputStream 与缓冲流的正确组合
- 理解字符编码问题:为什么中文乱码、怎么读 UTF-8 文件
- 掌握现代 NIO.2 API:Path/Files 读写、拷贝、遍历
- 掌握对象序列化与反序列化,理解 serialVersionUID 的作用
- 掌握 Properties 配置文件的读写
- 掌握生产级文件处理:try-with-resources、追加写、路径规范

💡 从 C 到 Java:文件世界的翻译

C 里的一切都是 `FILE*`:`fopen/fread/fwrite/fclose`。Java 里没有 `FILE*`,取而代之的是"流(stream)"体系:字节流(`InputStream/OutputStream`)按字节搬数据,字符流(`Reader/Writer`)按字符搬数据。而字符流底层还是字节流——中间垫着一层"编码转换器"。C 的 `printf/fprintf` 到 Java 是 `PrintStream/PrintWriter`。

最大的坑是编码。C 程序员在 Linux 上写文件,`char` 数组默认 UTF-8,基本没乱码问题;Java 里"读文件用什么字符集"是显式决策——`new FileReader(file)` 用平台默认字符集(Windows 上可能是 GBK!),读 UTF-8 文件直接乱码。**铁律:**读写文本一律显式指定 UTF-8。乱码问题 90% 由此而来。

NIO.2 是"现代姿势"。老 API(`File`、`FileInputStream`)啰嗦且容易漏关;JDK7 的 `Path / Files` 把"读整个文件、拷贝、移动、遍历"变成一行调用,配合 `try-with-resources` 几乎没有资源泄漏机会。生产代码应当以 NIO.2 为主,老 API 能读懂即可。

序列化是 C 没有的直接对应物。C 存结构体用 `fwrite` 裸写内存(字节序、版本问题一堆);Java 的 `Serializable` 把对象图整体变成字节流,再原样恢复。它不跨语言、有安全风险,但"Java 到 Java 的持久化/传输"非常方便——生产里常用于缓存与 RPC 传输(现代框架更推荐 JSON/Protobuf,本章 §7.7 讲选择)。

学习顺序:先建"流"的心智模型(§7.1-7.3),再学现代 API(§7.4),然后序列化与配置(§7.5-7.6),最后工程实践。

📦 前置知识自查

- C 文件操作:`fopen` 模式(`r/w/a`)、`fread/fwrite`、`fgets/fprintf`、`feof` 判断
- 第5章:`try-with-resources` 资源管理(本章全程使用)
- 第1章:`byte` 类型与 `char`/字符串概念;字节与字符的区别

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**§7.1-7.3 字节流/字符流与编码,动手制造一次乱码再修好
2. **第二阶段(约 1.5 小时):**§7.4 NIO.2,把老 API 的例子逐行改成 `Files` 写法
3. **第三阶段(约 1.5 小时):**§7.5 序列化 + §7.6 Properties,完成章末实验
4. **验收标准:**能写出"显式 UTF-8 读文本文件"的代码;能用 `Files` 拷贝文件;能序列化并恢复对象

本章知识导图

- §7.1 流的世界观:字节流 / 字符流 / 流包装
- §7.2 字节流实战:文件复制与缓冲
- §7.3 字符流与编码:Reader / Writer / UTF-8 铁律
- §7.4 NIO.2:Path / Files 现代 API
- §7.5 序列化与反序列化:serialVersionUID
- §7.6 Properties 配置文件
- §7.7 工程实践:追加日志 / 路径 / 格式选型

§7.1 流的世界观

7.1.1 四大基类与包装模式

方向	字节流(处理 byte)	字符流(处理 char/字符串)	C 对应
读	InputStream(FileInputStream 等)	Reader(FileReader 等)	fread / fgets
写	OutputStream(FileOutputStream 等)	Writer(Writer 等)	fwrite / fprintf

包装模式是 Java 流的灵魂:基础流只管"连接数据源/目的地",装饰流负责"加缓冲、转字符、按行读"。一个完整流 = 基础流 + 若干装饰流,最外层装饰流决定"用起来多方便"。读文本文件的经典组合:

```
FileInputStream -> InputStreamReader -> BufferedReader
```

对应到 C:FILE* 内部其实也是"系统 fd + 用户态缓冲"的两层结构,只是 C 帮你包好了;Java 把每一层都拆开让你选。

★ 重点:流的五个常识

- 流是 单向 的:读流不能写,写流不能读;要读写一个文件就开两个流
- 流必须 关闭 :try-with-resources 包住最外层流,内层自动关
- 读到最后返回 `-1` (字节流)或 `null` (readLine),不是异常
- 写满缓冲要 `flush()`:缓冲流把数据攒着,不 flush 可能丢尾巴
- 流是 顺序 访问:没有 fseek 的自由定位不是日常需求,需要时用 RandomAccessFile(少见)

§7.2 字节流实战:文件复制

示例 7-1 字节流文件复制(经典三段式)

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.IOException;

public class CopyBytes {
    public static void main(String[] args) throws IOException {
        // C 的 fopen("rb"/"wb") 对应:FileInputStream/FileOutputStream
        // 缓冲流:一次读 8KB 到内存,而不是逐字节系统调用
        try (var in = new BufferedInputStream(new FileInputStream("a.bin"));
            var out = new BufferedOutputStream(new FileOutputStream("b.bin"))) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) != -1) { // 读一坨;返回实际字节数
                out.write(buf, 0, n); // 只写实际读到的部分
            }
        } // 两个流自动关闭(先关 out 后关 in)
        System.out.println("复制完成");
    }
}

```

⚠ 易错点 1:write 一定要用 "buf, 0, n"

最后一次 read 可能只填满一部分缓冲区,用 `out.write(buf, 0, n)` 只写有效部分;写 `out.write(buf)` 会把上一次残留的脏字节也写进去,文件末尾出现垃圾数据。这是文件复制代码最高频的 bug。同理,read 的返回值 n 每次都要检查。

§7.3 字符流与编码

7.3.1 为什么乱码:字节与字符之间隔着一层"翻译官"

磁盘上只有字节。"中"在 UTF-8 里是 3 个字节 `E4 B8 AD`,在 GBK 里是 2 个字节 `D6 D0`。字符流就是"翻译官":读时按指定字符集把字节翻译成字符,写时反向翻译。翻译官选错(字符集不匹配) = 乱码。两个铁律:

1. 读文本文件: `FileReader` 必须显式传 UTF-8,否则用平台默认(Windows 常见 GBK/UTF-8 混乱)
2. 写文本文件:同一条铁律;写完 flush 或关闭

示例 7-2 字符流:显式 UTF-8 读 + 写

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class TextIo {
    public static void main(String[] args) throws IOException {
        Path path = Path.of("notes.txt");

        // 写法 1(老风格):手动组装装饰流
        try (var in = new BufferedReader(
            new java.io.InputStreamReader(
                new java.io.FileInputStream(path.toFile()),
                StandardCharsets.UTF_8))) {
            String line;
            while ((line = in.readLine()) != null) { // 读一行;结尾返回 null
                System.out.println("读到: " + line);
            }
        }

        // 写法 2(现代):Files.newBufferedReader 一行拿到 UTF-8 BufferedReader
        try (var in = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println("再读: " + line);
            }
        }

        // 写: newBufferedWriter 同样带字符集参数
        try (var out = Files.newBufferedWriter(path, StandardCharsets.UTF_8)) {
            out.write("第一行\n");
            out.write("第二行\n"); // 关流时自动 flush
        }
    }
}

```

⚠ 易错点 2:FileReader/FileWriter 的默认字符集陷阱

`new FileReader(file)` 与 `new FileWriter(file)` 用 **平台默认字符集**——在 Windows 上通常是 GBK,在 Linux 服务器上通常是 UTF-8。同一份代码两个平台行为不同,文件跨机器拷贝就乱码。生产规则:这两类直接禁用裸构造,要么用 `new InputStreamReader(in, StandardCharsets.UTF_8)`,要么用 `Files.newBufferedReader(path, UTF_8)`。字符集问题必须在编译时就写死,不能依赖环境。

§7.4 NIO.2:Path 与 Files

7.4.1 现代文件 API 全家桶

`Path` 描述位置(对应 C 的路径字符串,但类型安全),`Files` 是静态工具类(对应 C 的 `stat/rename/unlink` 等系统调用)。日常 90% 的文件操作 `Files` 都有现成方法:

操作	Files 方法	C 对应
读整个文件	Files.readString(path, UTF_8) / readAllLines	read + 循环
写整个文件	Files.writeString(path, text, UTF_8)	fwrite
拷贝	Files.copy(src, dst, REPLACE_EXISTING)	手工循环复制
移动/改名	Files.move(src, dst)	rename()
删除	Files.deleteIfExists(path)	unlink()
属性	Files.size / exists / isDirectory	stat()
遍历目录	Files.list / Files.walk(递归)	readdir 循环
创建目录	Files.createDirectories(可多级)	mkdir -p 的手工版

示例 7-3 Files 全家桶实战

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

public class FilesDemo {
    public static void main(String[] args) throws IOException {
        // 多级目录一次建好(等价 mkdir -p)
        Path dir = Path.of("data", "2026", "08");
        Files.createDirectories(dir);

        // 写整个文件:UTF-8 显式指定;文本默认不换行,自己加 \n
        Path f = dir.resolve("report.txt"); // resolve = 拼路径
        Files.writeString(f, "2026-08-26 实验报告\n第7章\n", StandardCharsets.UTF_8);

        // 读整个文件(小文件专用)
        String content = Files.readString(f, StandardCharsets.UTF_8);
        System.out.println("内容:\n" + content);

        // 追加写:Append 参数
        Files.writeString(f, "追加一行\n", StandardCharsets.UTF_8,
            java.nio.file.StandardOpenOption.APPEND);

        // 拷贝 + 覆盖
        Path bak = dir.resolve("report.bak.txt");
        Files.copy(f, bak, StandardCopyOption.REPLACE_EXISTING);

        // 遍历目录(非递归)
        try (var stream = Files.list(dir)) {
            stream.forEach(p -> System.out.println("条目: " + p.getFileName()));
        }

        // 递归遍历
        try (var walk = Files.walk(Path.of("data"))) {
            walk.filter(p -> p.toString().endsWith(".txt"))
                .forEach(p -> System.out.println("txt 文件: " + p));
        }

        // 清理:删除备份
        Files.deleteIfExists(bak);
        System.out.println("done");
    }
}
```

工程建议:Files 的边界

`readString/readAllLines` 只适合小文件 (几 MB 以内);大文件必须流式处理(\$7.2 的缓冲流或 `Files.lines`)。 `Files.list` 返回的是流,必须 `try-with-resources` 关闭,否则句柄泄漏。另外 `Path.of` 与 `Paths.get` 等价,新代码用 `Path.of`。

§7.5 序列化与反序列化

7.5.1 对象 → 字节流 → 对象

类实现 `java.io.Serializable` (空接口,标记用),对象就能交给 `ObjectOutputStream` 写成字节,再用 `ObjectInputStream` 原样读回。 `serialVersionUID` 是版本号:类结构变了(增删字段),UID 变了,老数据反序列化会抛 `InvalidClassException`。生产铁律:每个 `Serializable` 类都显式声明 `serialVersionUID`,不依赖编译器生成。

示例 7-4 序列化与版本控制

```
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;
import java.io.Serializable;
import java.nio.file.Files;
import java.nio.file.Path;

public class SerializeDemo {
    // 实现 Serializable 标记接口:不写任何方法
    static class Student implements Serializable {
        private static final long serialVersionUID = 1L;    // 显式版本号:铁律!

        final String name;
        final int score;

        Student(String name, int score) {
            this.name = name;
            this.score = score;
        }

        @Override
        public String toString() { return name + ":" + score; }
    }

    public static void main(String[] args) throws Exception {
        Path f = Path.of("student.dat");

        // 写:对象 → 字节流
        try (var out = new ObjectOutputStream(
            Files.newOutputStream(f))) {
            out.writeObject(new Student("张三", 88));
            out.writeObject(new Student("李四", 76));
        }

        // 读:字节流 → 对象(顺序必须与写一致)
        try (var in = new ObjectInputStream(
            Files.newInputStream(f))) {
            Student s1 = (Student) in.readObject();    // 需要强转
            Student s2 = (Student) in.readObject();
            System.out.println(s1 + " / " + s2);        // 张三:88 / 李四:76
        }
    }
}
```

⚠ 易错点 3:反序列化的两大风险

① 安全风险:反序列化等于执行"数据里的指令",从不信任的来源(网络、用户上传)反序列化会被攻击者利用(经典 RCE 链)。生产原则:反序列化只用于"自己写、自己读"的数据;跨系统传输用 JSON(Jackson/Gson)。② 兼容性:字段增删会让旧数据读不出来——serialVersionUID 不变时,新增字段读回默认值(null/0),删除字段的数据会丢;对格式敏感的场景考虑版本字段或 JSON。

§7.6 Properties 配置文件

7.6.1 键值对配置的标准读法

Properties 读写 `key=value` 格式的 .properties 文件,是 Java 最传统的配置方式(现代项目也用 YAML,但 Properties 依然到处可见)。加载后用 `getProperty` 取配置,带默认值防 NPE。

示例 7-5 Properties 读写

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Properties;

public class ConfigDemo {
    public static void main(String[] args) throws IOException {
        Path cfg = Path.of("app.properties");

        // 写配置(演示;生产配置一般由运维/安装器生成)
        Properties p = new Properties();
        p.setProperty("server.port", "8080");
        p.setProperty("app.name", "教务系统");
        try (var out = Files.newBufferedWriter(cfg, StandardCharsets.UTF_8)) {
            p.store(out, "生成于演示"); // 注释头 + 键值对
        }

        // 读配置:标准生产姿势
        Properties loaded = new Properties();
        try (var in = Files.newBufferedReader(cfg, StandardCharsets.UTF_8)) {
            loaded.load(in);
        }
        int port = Integer.parseInt(
            loaded.getProperty("server.port", "8080")); // 带默认值
        String name = loaded.getProperty("app.name", "未知");
        System.out.println("端口=" + port + ", 应用=" + name);
        // 取不到的键返回 null;getProperty(key, def) 给默认值
    }
}
```

🚀 工程建议:配置的读取时机

配置在启动时读一次,放进常量/不可变对象,不要在每次请求时重新读文件(慢且浪费)。生产配置常配合环境变量覆盖:代码读系统属性/环境变量优先,Properties 兜底: `System.getenv("PORT")` 存在就用它。配置中心(Nacos/Spring Cloud Config)本质还是"远程拉一份配置到本地再走这套逻辑"。

§7.7 工程实践

7.7.1 追加日志与路径规范

- **追加写日志:** `Files.writeString(path, line, UTF_8, StandardOpenOption.APPEND, CREATE)` ——多线程追加用日志框架(logback),别自己并发写文件

- **路径规范:**用 Path 拼接而不是字符串 + (Windows 分隔符 \ 与 Linux / 的差异由 Path 处理); .. 处理用 normalize
- **相对路径依赖 cwd:**生产用绝对路径或配置注入,不要依赖"当前目录"

7.7.2 格式选型速查

场景	选型	理由
Java 内部持久化/缓存	序列化 / 新版用 record	对象图整体保存,最省事
跨语言/对外 API	JSON(Jackson)	人类可读、生态最大、安全
高性能二进制	Protobuf / 消息队列自带	体积小、解析快、带 schema
简单键值配置	Properties / YAML	人写人读
大文件流式处理	缓冲流 + 循环	不占内存

本章速查表

主题	要点
四大基类	InputStream/OutputStream(字节)、Reader/Writer(字符);流单向、要关闭
装饰流	BufferedInputStream/BufferedReader 等;外层包内层;try-with-resources 包最外层
读文件	read 返回 -1 结束;readLine 返回 null 结束;write 必须用 write(buf, 0, n)
编码铁律	文本读写一律显式 StandardCharsets.UTF_8;禁用裸 FileReader/FileWriter
NIO.2	Path.of / Files.readString / writeString / copy / move / walk;小文件专用
大文件	缓冲流 + 8KB 数组循环;Files.lines 流式
序列化	implements Serializable + 显式 serialVersionUID;读写顺序一致;不信任来源禁反序列化
Properties	load/store;getProperty(key, 默认值) 防 NPE;启动时读一次
追加写	Files.writeString(path, s, UTF_8, APPEND, CREATE)
路径	用 Path 拼接,不用字符串 +;生产用绝对路径或配置注入
格式选型	Java 内部→序列化;跨语言→JSON;高性能→Protobuf
乱码排查	确认文件实际编码(hexdump 看字节)→ 确认读写字符集一致 → 修改代码显式 UTF-8

最小必会清单

- 能画出"FileInputStream → InputStreamReader → BufferedReader"的包装链并解释每层职责
- 能写出字节流文件复制代码(含 write(buf, 0, n) 的正确姿势)
- 能解释乱码成因,并写出显式 UTF-8 读写文本的两行代码
- 能用 Files 完成:读整个文件、写整个文件、拷贝、遍历目录
- 能写出序列化类(含 serialVersionUID)并说出安全红线
- 能读写 Properties 配置并说出默认值参数的用途

自测题

Q 1. 读一个 UTF-8 文件出现乱码,可能的原因有哪些?排查顺序?

 **参考答案**

原因:① 读取字符集不是 UTF-8(如 FileReader 用了平台默认 GBK);② 文件本身不是 UTF-8(GBK/ANSI 编码);③ 多次转换叠加损坏。排查:先用工具确认文件字节(如 hexdump 看"中"是 E4B8AD 还是 D6D0),再确认代码里字符集参数,最后统一为显式 UTF-8 读写。乱码一旦写回文件就是不可逆损坏,务必先备份。

Q 2. 下面代码有什么问题? `byte[] buf = new byte[4096]; in.read(buf); out.write(buf);`

✓ 参考答案

read 的返回值没检查,且 write 写整个 buf。最后一次 read 可能不足 4096 字节,write(buf) 会把上次残留内容一起写进去,文件尾部出现垃圾。正确写法: `int n; while ((n = in.read(buf)) != -1) out.write(buf, 0, n);`

Q 3. 序列化版本兼容:类加了 `serialVersionUID = 1L` 后新增一个字段,读旧数据会怎样?为什么显式声明 UID?

✓ 参考答案

UID 不变时,新增字段读回默认值(引用 null、数字 0),不抛异常——这是"宽松兼容";删除字段的数据会丢失。若没显式声明 UID,编译器根据类结构自动生成,增删字段后 UID 变化,读旧数据抛 `InvalidClassException`。所以显式声明让版本行为可预期,并且改版时你能主动决定是否 bump 版本。

Q 4. 从不可信来源(用户上传文件)反序列化,为什么危险?

✓ 参考答案

反序列化会实例化数据中指定的类并调用其方法,攻击者可以构造恶意字节流触发危险类(如 Runtime、JNDI 相关)执行任意代码——著名的反序列化 RCE。生产铁律:只对可信数据反序列化;跨系统用 JSON 并校验字段白名单。

Q 5. 需要把 2GB 的日志文件按行统计行数,用 `readString` 还是流式?写出代码。

✓ 参考答案

```
long count = 0;
try (var lines = Files.lines(Path.of("big.log"), StandardCharsets.UTF_8)) {
    count = lines.count();
}
System.out.println("行数: " + count);
```

必须流式:readString 会把 2GB 塞进内存直接 OOM。Files.lines 返回惰性流,一行一行读,内存占用与文件大小无关。同理大文件处理用 stream() 的过滤/映射,别先读全量。

Q 6. 追加日志时用 `writeString` 加 APPEND,多个线程同时写会怎样?正确做法?

✓ 参考答案

并发写同一文件会互相覆盖/交错,出现损坏或丢行。生产用日志框架(logback/log4j2)管理文件追加、滚动与并发;实在要手写,用单线程写队列或加锁,但那是重复造轮子。多线程与锁在本书第9章展开。

章末实验题:成绩单文件处理器

实验 7:成绩单文件处理器

实验目标:编写一个成绩单处理程序,综合运用本章全部知识点:字节流、字符流、编码、NIO.2、序列化、Properties。

实验要求:

- 任务 1:程序启动时用 Properties 读取 config.properties(端口无关,读报告标题与输出目录)(覆盖 §7.6 配置)
- 任务 2:用 Files 读取 CSV 成绩文件 scores.csv(UTF-8,每行"姓名,分数"),按行解析(覆盖 §7.4 NIO.2 读)
- 任务 3:计算平均分、最高分、及格人数,用 Files.writeString 生成 report.txt 报告(覆盖 §7.4 NIO.2 写)
- 任务 4:用缓冲字符流把原始成绩逐行"复制"成 backup.csv 备份(覆盖 §7.2-7.3 流式复制)
- 任务 5:定义 Student 类(含 serialVersionUID),把解析结果整体序列化到 students.dat,再读回验证(覆盖 §7.5 序列化)
- 任务 6:用 Files.copy 把报告复制一份到备份目录,并列出输出目录全部文件(覆盖 §7.4 拷贝/遍历)
- 任务 7:输出完整处理报告(覆盖全部)

实验环境:JDK 17+;先手工创建 scores.csv(至少 5 行数据,注意存成 UTF-8)。

第一步 写数据类 Student 与 CSV 解析,再用 System.out 验证解析正确,再落盘

第二步 务 5 注意:Student 要实现 Serializable 且两个字段建议 final

第三步 有流都包 try-with-resources;写完立即验证文件内容(打开看看)

参考答案要点

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;

/** 成绩单文件处理器 — 第7章综合实验参考答案
 * 覆盖:Properties、NIO.2 读写、流式复制、序列化、拷贝与遍历
 */
public class ScoreFileProcessor {

    // 任务 5:序列化数据类型,显式版本号
    static class Student implements Serializable {
        private static final long serialVersionUID = 1L;

        final String name;
        final int score;

        Student(String name, int score) {
            this.name = name;
            this.score = score;
        }

        @Override
        public String toString() { return name + ":" + score; }
    }

    public static void main(String[] args) throws Exception {
        // 任务 1:读取配置(带默认值)
        Properties cfg = new Properties();
        if (Files.exists(Path.of("config.properties"))) {
            try (var in = Files.newBufferedReader(Path.of("config.properties"),
                StandardCharsets.UTF_8)) {
                cfg.load(in);
            }
        }
        String title = cfg.getProperty("report.title", "期末成绩报告");
        String outDir = cfg.getProperty("output.dir", "out");
        Path out = Path.of(outDir);
        Files.createDirectories(out); // 目录不存在就建
        System.out.println("== 任务 1 配置: " + title + " / " + outDir);

        // 任务 2:Files 读 CSV 并解析
        List<Student> students = new ArrayList<>();
        try (var reader = Files.newBufferedReader(Path.of("scores.csv"),
            StandardCharsets.UTF_8)) {
            String line;
            while ((line = reader.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) {
                    continue; // 跳过空行与注释
                }
                String[] parts = line.split(",");
                if (parts.length != 2) {
                    System.out.println("跳过格式错误行: " + line);
                    continue;
                }
                students.add(new Student(parts[0].trim(),
                    Integer.parseInt(parts[1].trim())));
            }
        }
    }
}

```

```

    }
    System.out.println("== 任务 2 解析 " + students.size() + " 条成绩");

    // 任务 3:统计并写报告
    int sum = 0, pass = 0, max = 0;
    for (Student s : students) {
        sum += s.score;
        if (s.score >= 60) pass++;
        if (s.score > max) max = s.score;
    }
    double avg = students.isEmpty() ? 0 : (double) sum / students.size();
    StringBuilder report = new StringBuilder();
    report.append("=== ").append(title).append(" ===\n");
    report.append("人数: ").append(students.size()).append("\n");
    report.append(String.format("平均分: %.1f\n", avg));
    report.append("最高分: ").append(max).append("\n");
    report.append("及格人数: ").append(pass).append("\n");
    Path reportFile = out.resolve("report.txt");
    Files.writeString(reportFile, report.toString(), StandardCharsets.UTF_8);
    System.out.println("== 任务 3 报告已生成: " + reportFile);

    // 任务 4:缓冲字符流逐行备份(流式,不整文件进内存)
    Path backup = out.resolve("backup.csv");
    try (var reader = Files.newBufferedReader(Path.of("scores.csv"),
        StandardCharsets.UTF_8);
        var writer = Files.newBufferedWriter(backup, StandardCharsets.UTF_8)) {
        String line;
        while ((line = reader.readLine()) != null) {
            writer.write(line);
            writer.newLine();
        }
    }
    System.out.println("== 任务 4 备份完成: " + backup);

    // 任务 5:序列化整个列表,再读回验证
    Path dat = out.resolve("students.dat");
    try (var oos = new ObjectOutputStream(Files.newOutputStream(dat))) {
        oos.writeObject(students);
    }
    try (var ois = new ObjectInputStream(Files.newInputStream(dat))) {
        @SuppressWarnings("unchecked")
        List<Student> restored = (List<Student>) ois.readObject();
        System.out.println("== 任务 5 反序列化验证: " + restored.size() + " 条,"
            + "首条 " + restored.get(0));
    }

    // 任务 6:拷贝报告 + 遍历目录
    Path copy = out.resolve("report.backup.txt");
    Files.copy(reportFile, copy, StandardCopyOption.REPLACE_EXISTING);
    System.out.println("== 任务 6 输出目录内容:");
    try (var list = Files.list(out)) {
        list.sorted().forEach(p ->
            System.out.println("    " + p.getFileName() + " (" + sizeOf(p) + " B)"));
    }

    System.out.println("== 任务 7 处理完成 ==");
}

static long sizeOf(Path p) {
    try {
        return Files.size(p);
    } catch (Exception e) {
        return -1;
    }
}
}

```

设计说明:任务 4 特意用流式逐行复制(与任务 2 的整文件读形成对照):CSV 如果很大,readString 会 OOM,而这里内存占用恒定。任务 5 的 @SuppressWarnings("unchecked") 是强转泛型列表的标准姿势。扩展方向:用 Files.walk 递归处理子目录、把统计结果同时写 JSON 版本(第13章引入第三方库时会讲)、对超大文件用 Stream 并行统计(第10章)。

下一章预告:第8章 网络通信。Socket 编程与 HttpClient——TCP 的 Java 封装、HTTP 客户端、网络编程的坑与安全红线。